Carleton University

School of Computer Science

COMP 4905 — Honours Project

SLAM From Scratch

Submitted by:

Mario Pardo

Supervisor:

Mark Lanthier

April 2026

Abstract

Humans are able to navigate and understand new environments quickly and intuitively, combining sensory inputs with spatial memory to create an understanding of their environment and their location within it. Replicating this in machines is a complicated task encompassing various areas of research and is a key challenge in developing autonomous robots and vehicles. Simultaneous Localization and Mapping (SLAM) is a class of algorithms dedicated to just this task, allowing systems to understand the environment in which they operate and where they are within it — especially critical in scenarios where external positioning sources, such as GPS, are unavailable.

This project presents a C++ implementation of a Graph-Based SLAM system developed from first principles. The resulting system demonstrates reliable convergence in pose-graph optimization, validating that a ground-up implementation can manage the underlying mathematical complexity and high-performance requirements of modern robotics.

Acknowledgments

Acknowledgments to be completed.

Contents

1 Introduction **1**

2 System Architecture **2**

2.1 Technology Stack 3

3 World Setup and Sensor Integration **4**

3.1 Arena and Robot Setup 4

3.2 Odometry 4

3.3 Point Cloud 5

3.4 Webots Lidar Noise During Rotation 5

4 Occupancy Grid **6**

5 ICP: Scan Matching **7**

5.1 Correspondence Matching 7

5.2 Estimating the Transform 8

5.3 Convergence 8

5.4 Handling Rotation 8

5.5 ICP 180-Degree Inversion Bug 9

6 Pose Graph Construction **10**

7 Loop Closure Detection **11**

7.1 Performance Tuning 12

8 Pose Graph Optimization **13**

8.1 Problem Formulation 13

8.2 Solving via Iterative Local Linearization 13

8.3 Implementation 14

9 Results **15**

9.1 Trajectory Accuracy 15

9.2 Mapping Accuracy 16

9.3 Pose Graph Optimization 17

9.4 Validation on a Real-World Dataset 17

9.5 Future Work 18

References **19**

A Representative Source Code **20**

List of Figures

2.1 Visualization of a the data flow and main components of the Pose Graph SLAM System. 2

3.1 Initial visualizations showing working trajectory estimation using odometry, and lidar point cloud in robot frame 5

5.1 Initial working ICP implementation, still dealing with mentioned rotation smear 9

6.1 Diagram of Pose Graph and it's meaning 10

7.1 Visualization of a detected loop closures (cyan) and the resulting graph edge connecting the current pose to a previously visited node. 11

9.1 Final Trajectories and Pose Graph in simulation environment 15

9.2 In progress occupancy grid from Odometry(left) and ICP(right) 16

9.3 Ground truth vs. SLAM occupancy grid output for the simulation environment. 16

9.4 Ground truth vs. SLAM output on the Intel Research Lab dataset. . . . 17

List of Tables

9.1 Pose graph optimization results across multiple test runs. 17

1. Introduction

Robotic systems sense the world and themselves through a variety of sensors, such as wheel encoders, gyroscopes, and lidar sensors. The two former are combined to create *odometry* data, estimating the change in position and orientation (pose), while the latter uses laser reflections to sense the geometry of the world at a distance. In real-world settings, odometry data carries noise from sensor readings and misalignments between these readings and the motion they imply — for example, wheels slipping over bumps. This causes *drift*: an error in the calculated pose compared to the actual pose. As pose estimates build on top of one another, this drift accumulates quickly, leading to worsening estimates and conflicts with incoming sensor data — ultimately producing catastrophic errors in localization and mapping.

To illustrate: lidar data might indicate a wall 3 m directly ahead, but the pose estimate says the robot is at (3 m, 1 m) while it is in reality at (2 m, 0 m). The system then believes the wall is at (6 m, 1 m) when it is actually at (5 m, 0 m).

A Graph-Based SLAM system constructs a graph of poses and movements, and when it detects a *loop closure* — a location previously visited — it uses this new observation to update all previous pose estimates, "snapping" both trajectory and map into a corrected version.

The intention of this project was to open this algorithmic black box and gain rigorous experience studying, understanding, implementing, debugging, and testing a complex system from first principles. Developing a SLAM algorithm tested knowledge in mathematics — specifically linear algebra and statistics — and its application to a complex, multi-step, real-world system written in high-performance C++.

2. System Architecture

A SLAM system starts with sensor inputs and, over time, accumulates information to locate itself and map its surroundings. It is composed of a chain of subsystems of varying complexity, each depending on the step before it to produce meaningful results.

The system can be broken down into 4 main stages:

1. **Sensor Integration**: Collect sensor data and process into usable odometry data and point clouds
2. **Transformation Estimation:** Iterative Closest Point (ICP) to estimate robot motion and map surrounding environment
3. **Pose Graph Construction**: Keep track of trajectory and mapping data through the construction of a Pose Graph
4. **Loop Closures and Pose Graph Optimization**: If robot has recognized a familiar pose, try to correct error in pose graph

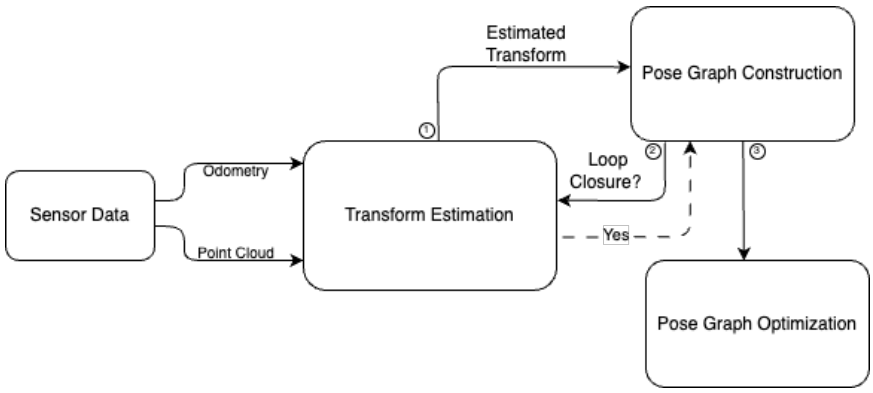


Figure 2.1: Visualization of a the data flow and main components of the Pose Graph SLAM System.

2.1 Technology Stack

C++ — Core Algorithm. C++ continues to be the language of choice for high-performance code. Writing the backend in C++ provided an excellent opportunity to make use of its powerful features and meet the intense performance requirements of robotics software. Key libraries used:

- **Eigen** — linear algebra.
- **GTSAM** — pose graph optimization.

Webots — Robotics Simulation. Webots is a simple yet powerful open-source robotics simulation platform. It provides ready-made sandboxes and robot models, such as the TurtleBot Burger 3 used in this project, with the necessary setup for motion and sensor usage. Using these ready-made tools allowed focus on the core SLAM implementation and allows for easy scaling of the project.

Python — Visualization and Communication. Python was used on both sides of visualization: interfacing with Webots for robot simulation, and on the backend for visualizing trajectories, occupancy grids, and system outputs. It was selected for its ease of use and wide library support.

ZeroMQ — Communication Protocol. ZeroMQ was selected for communication between the front end, backend, and visualization tools. It implements the Publish/Subscribe architecture, which is well suited to a use case where systems must communicate consistently in both directions without requiring full synchronization. The name reflects its need for no broker or central server, making it resilient to any single endpoint failing, and it is straightforward to implement from both C++ and Python.

3. World Setup and Sensor Integration

Various sensors in the robot acquire data at each timestep. This project makes use of:

- **Wheel encoders** — measure the rotation of each wheel, processed into odometry data that estimates the robot's displacement.
- **Lidar** — a 360° scan of the environment using laser light; the reflection of each projected beam records the distance to the nearest surface at a given angle.

This data is gathered in Webots and the raw values are forwarded to the C++ backend for processing.

3.1 Arena and Robot Setup

The first step was to set up the testing arena and robot. A simple square arena was created that is easy to interpret in visualization tools while containing sufficient detail and landmarks for ICP (Chapter 5) to work with. The premade TurtleBot Burger 3 was selected, already equipped with all necessary sensors. A simple differential drive controller was created and tested to confirm reliable movement.

3.2 Odometry

Using the known wheel diameter and wheelbase width, the robot's trajectory is calculated from wheel sensor data.

Noise. In a real-world setting, wheel sensors are not perfect. They can record movement that does not result in actual robot displacement — such as wheel slip on wet terrain — or introduce sensor measurement noise. Modelling this noise is important, as correcting for it is a core responsibility of the SLAM system.

Odometry provides the first reference point for the SLAM algorithm. The robot's position was first estimated using odometry alone as a crucial validation step, confirming that the trajectory can be accurately calculated before introducing additional components.

3.3 Point Cloud

Using simple trigonometry, for each beam from the lidar sensor the landing point — and therefore the location of the nearest surface — can be calculated from the robot's frame of reference.

World Frame Projection. With the estimated current robot pose, the point cloud is projected into a static world frame, providing an estimate of where surfaces lie on the map. An initial visualization tool was implemented to display the robot's trajectory alongside the surfaces currently observed. The tool was subsequently upgraded (indicated visually by a dark background) to maintain a history of all previously observed surfaces, yielding a persistent map of the environment.

3.4 Webots Lidar Noise During Rotation

Even after introducing rotation gating for localization (described in Section 5.4), significant map smear persisted. After considerable troubleshooting, the source was identified as a Webots simulator issue: while the robot is rotating, the lidar scan twitches back and forth and only snaps back into place once rotation finishes, producing very noisy and incorrect lidar data during the rotation. The fix was to discard all lidar data collected while the robot is rotating, using only readings when the robot is undergoing no rotation, where lidar readings are accurate

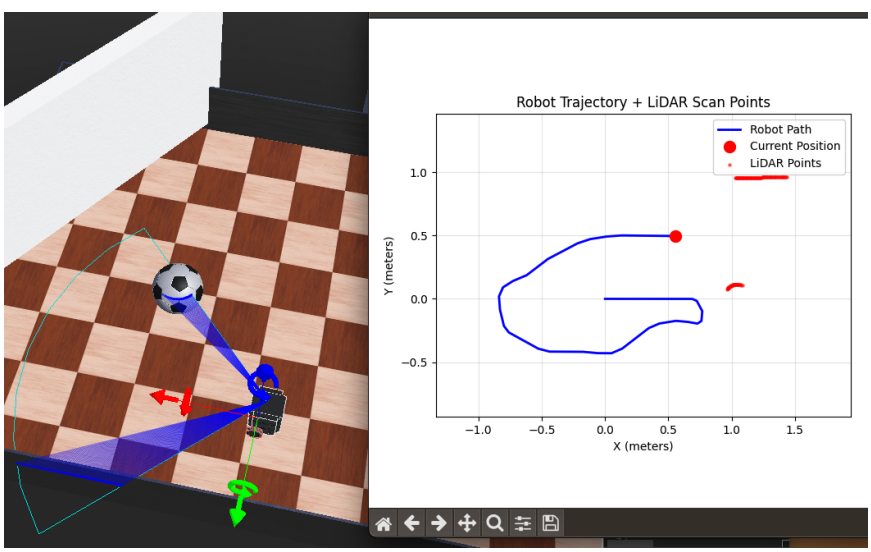


Figure 3.1: Initial visualizations showing working trajectory estimation using odometry, and lidar point cloud in robot frame

4. Occupancy Grid

To accomplish the mapping portion of SLAM, an occupancy grid represents the map by storing the likelihood that each cell is occupied, encoded using log-odds:

$$\ell(m_k) = \log \frac{P(m_k \mid z_{1:t},\ x_{1:t})}{1 - P(m_k \mid z_{1:t},\ x_{1:t})} \tag{4.1}$$

where m_k is the k-th cell, $z_{1:t}$ is the history of sensor readings, and $x_{1:t}$ is the history of poses. As the robot receives lidar returns, the likelihood of occupied cells along each beam is increased, and cells along the free path are decreased.

The occupancy grid is only updated on new keyframes (Chapter 6) to ensure readings come from stable and trusted data.

5. ICP: Scan Matching

Iterative Closest Point (ICP) is one of the most crucial components of the SLAM system. It takes two point cloud readings — a previous scan and a current scan — and estimates how the robot has moved between them.

The algorithm, at a high level, proceeds as follows:

alignPointClouds(`sourcePointCloud`, `targetPointCloud`, `odometryInitialGuess`)

1. Transform the source point cloud by the initial guess provided by odometry.
2. **Find correspondences**: identify which points in the source and target clouds represent the same surface.
3. Using these correspondence pairs, **estimate the transformation** between the two clouds.
4. Update the current estimate with the refined transformation.
5. If the change in error falls below a threshold, ICP has **converged**.

5.1 Correspondence Matching

The most simple method of correspondence matching was used. The logic goes as follows for each point in the source cloud, the closest point in the target cloud is found and the pair is considered a correspondence.

A key hyperparameter is `max_distance`, the maximum distance at which two points can be paired. This is a trade-off between precision (smaller distance yields more accurate transformations) and generality (too small a distance results in too few matches).

5.2 Estimating the Transform

Given the set of correspondences, the transformation is estimated using the Singular Value Decomposition (SVD) method.

Let μ_s and μ_t denote the centroids of the source and target point sets respectively. Let $W = \sum_i (p_i - \mu_s)(q_i - \mu_t)^\top$ be their cross-covariance. Taking their outer product tells us how each pair of axes from each group correlates.

The SVD of W is given by $W = UDV^\top$. We can interpret V as the directions coming **into** the transformation, U as the transformed **output** direction vectors for those same inputted directions, and D as the scaling for each direction.

From this, we get the optimal rotation:

$$R = VU^\top \tag{5.1}$$

And then to get the full transformation, we find the translation from the rotated source centroid to the output centroid:

$$t = \mu_t - R\,\mu_s \tag{5.2}$$

Since a straightforward analytical solution is known, implementation was a matter of correctly converting these equations into code, verified by unit tests.

5.3 Convergence

The error between successive transformation estimates is computed. If it falls below a threshold, ICP is considered converged. This required hyperparameter tuning: errors were printed across many runs, the best 10% were identified, and the threshold was iteratively narrowed to one that gave consistent yet precise convergences.

5.4 Handling Rotation

Large rotations cause point clouds to change drastically from little robot movement, so ICP inherently struggles to find good transformations during rapid rotation. To address this, *gating* was introduced: if the robot's angular velocity exceeds a threshold, the odometry-estimated transformation is used instead of running ICP. This is a simple and effective fix, though a production system would use more sophisticated sensor fusion such

as an Extended Kalman Filter to intelligently weight sensor sources when any one cannot be fully relied upon.

Expanding the lidar field of view from 90° to 360° was a major breakthrough for ICP accuracy and consistency. With a 90° FOV, even small rotations shift a large fraction of the visible point cloud, making correspondence matching difficult. With a full 360° view, surfaces all around the robot remain visible throughout rotation, providing ICP with stable correspondences.

5.5 ICP 180-Degree Inversion Bug

Early in testing, ICP produced transformations at 180° from the actual movement. The initial patch manually rotated the output by 180°, which worked but was clearly a symptom of a deeper issue.

The root cause was a source/target mismatch. Consider a robot at $(0, 0)$ with a wall at $(0, 2)$. From the robot's frame, the wall is 2 m ahead. After the robot moves forward 1 m to $(0, 1)$, the wall appears 1 m ahead: from the robot's frame the wall has moved by $(0, -1)$. This is geometrically correct from the robot's point of view, but produces an inverted transformation.

To correct this, the source and target must be swapped. If the source point is at relative position $(0, 1)$ and the target at $(0, 2)$, the transformation found is $(0, +1)$, which correctly describes the robot's forward motion.

Figure 5.1: Initial working ICP implementation, still dealing with mentioned rotation smear

6. Pose Graph Construction

The pose graph is the record of where the robot has been and how the world has appeared at each timestep.

Graph Nodes contain:

- **Pose** — estimated position (x, y) and heading angle θ.
- **Point cloud** — how the world appeared at this pose.
- **ID and Timestamp** — for identification and ordering.

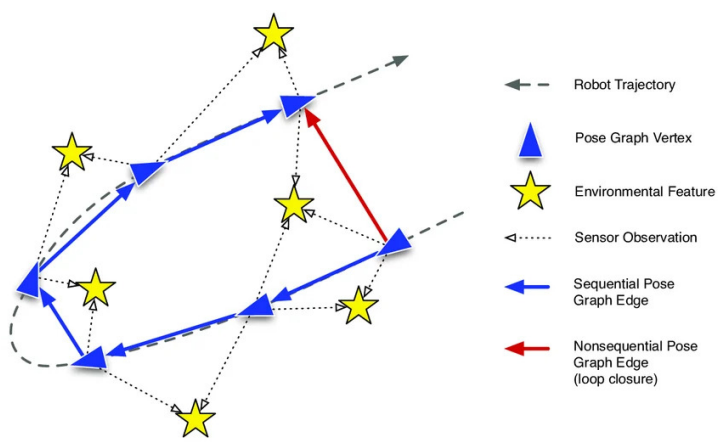


Figure 6.1: Diagram of Pose Graph and it's meaning

Edges represent the robot movement between nodes as calculated by ICP

Nodes (keyframes) are added only once the robot has moved a minimum distance d or rotated a minimum angle $\delta\theta$, tuned through trial and error. This is a trade-off: too many nodes overwhelm the optimizer; too few forces ICP to find transformations across large gaps. For a square world nearly 2 m diagonally across, a keyframe step of 15 cm was large enough not to overwhelm the optimizer while small enough for ICP to find accurate transformations.

The visualization tool was updated to display graph nodes and their ICP-estimated edges.

7. Loop Closure Detection

As the robot traverses the world and adds keyframes, the current position is compared with old graph nodes to detect whether a previously visited location has been returned to. An exact match is not required; being within a sufficient proximity is enough. If a candidate is found, ICP is run against the old node's point cloud to confirm the loop closure and obtain a precise relative pose estimate. A special *loop closure edge* is then added between the current node and the matched old node.

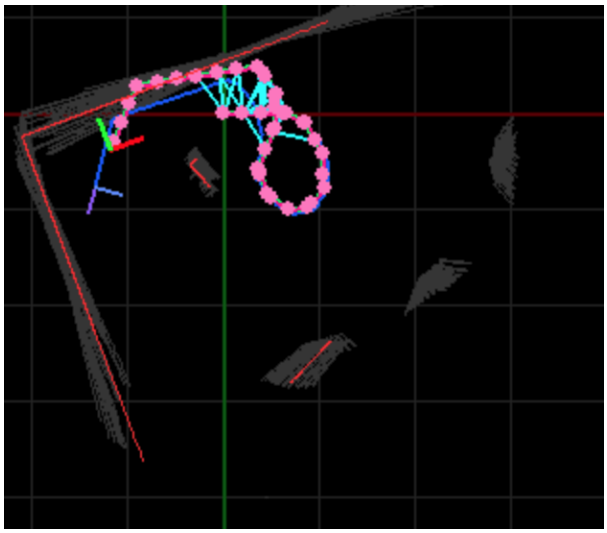


Figure 7.1: Visualization of a detected loop closures (cyan) and the resulting graph edge connecting the current pose to a previously visited node.

If no good match is found, or ICP does not converge on a consistent transformation, the robot continues accumulating keyframes as normal.

While loop closures reuse much of the ICP infrastructure, they are a crucial step: they determine when and how the graph is optimized, and therefore when accumulated error is corrected.

Hyperparameter tuning was required, notably for correspondence distance and error epsilon. The goal is accurate loop closure edges that are also general enough to reliably trigger when revisiting a location. Too selective, and loop closures never trigger; too permissive, and false closures corrupt the graph.

7.1 Performance Tuning

Loop closure detection requires running ICP repeatedly — a computationally expensive process — so the set of candidate nodes is filtered aggressively before ICP is invoked:

- **Recency filter.** Useful loop closures only involve nodes from well before the current position. Only nodes at least $N = 10$ keyframes back are considered.
- **Proximity filter.** A loop closure requires physical proximity. The 6 closest candidate nodes by Euclidean distance are retained before running ICP.
- **Temporal gating.** Once a loop closure has been added, subsequent closures in the same vicinity provide diminishing benefit and can challenge the optimizer. Loop closure attempts are therefore gated to a minimum of 3 keyframes after the last successful closure, providing significant computational savings without limiting system effectiveness.

8. Pose Graph Optimization

With a loop closure edge added, the graph forms a full loop and is ready to be optimized. Optimization finds the arrangement of node poses that minimizes the total error across all edges. This step is what "snaps" the map in place, through the reduction of acculumated drift

8.1 Problem Formulation

Let $\mathbf{x} = \{x_1, x_2, \ldots, x_n\}$ denote the state: the set of node poses. Let $\hat{z}_{ij}$ denote the *measured relative pose* between nodes i and j, as estimated by ICP. Let $z_{ij}(\mathbf{x})$ denote the *predicted relative pose*, computed from the difference in stored node coordinates.

The error for edge (i, j) is:

$$e_{ij}(\mathbf{x}) = z_{ij}(\mathbf{x}) - \hat{z}_{ij} \tag{8.1}$$

Each edge carries an information matrix Ω_{ij}, representing confidence in the measurement (derived from how much the ICP result differed from the odometry prediction). The weighted squared error per edge is:

$$\varepsilon_{ij}(\mathbf{x}) = e_{ij}(\mathbf{x})^\top \, \Omega_{ij} \, e_{ij}(\mathbf{x}) \tag{8.2}$$

Thus, to minimize error, the optimization goal is:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} \sum_{(i,j)\in\mathcal{E}} e_{ij}(\mathbf{x})^\top \, \Omega_{ij} \, e_{ij}(\mathbf{x}) \tag{8.3}$$

8.2 Solving via Iterative Local Linearization

This nonlinear least-squares problem is solved using iterative local linearization, upon which the Levenberg–Marquardt optimizer builds.

Step 1 — Linearize around the current estimate.

$$e_{ij}(\mathbf{x} + \Delta\mathbf{x}) \approx e_{ij}(\mathbf{x}) + J_{ij} \, \Delta\mathbf{x} \tag{8.4}$$

where J_{ij} is the Jacobian of e_{ij} with respect to $\mathbf{x}$. Linearizing transforms a curved error landscape into a locally flat one that can be solved with linear algebra.

Step 2 — Expand the squared error in terms of $\Delta\mathbf{x}$.

$$\begin{aligned}\varepsilon_{ij} &= \big(e_{ij} + J_{ij}\Delta\mathbf{x}\big)^\top \Omega_{ij} \big(e_{ij} + J_{ij}\Delta\mathbf{x}\big) \\ &= \underbrace{e_{ij}^\top \Omega_{ij} e_{ij}}_{C_{ij}} + 2\,\underbrace{e_{ij}^\top \Omega_{ij} J_{ij}}_{b_{ij}^\top}\,\Delta\mathbf{x} + \Delta\mathbf{x}^\top\,\underbrace{J_{ij}^\top \Omega_{ij} J_{ij}}_{H_{ij}}\,\Delta\mathbf{x}\end{aligned} \tag{8.5}$$

where $b_{ij} = J_{ij}^\top \Omega_{ij}\, e_{ij}$ encodes the gradient direction and $H_{ij} = J_{ij}^\top \Omega_{ij} J_{ij}$ is the Gauss–Newton Hessian approximation.

Step 3 — Differentiate, set to zero, and solve for $\Delta\mathbf{x}$. The GTSAM solver handles this step internally, iterating until convergence.

8.3 Implementation

GTSAM is a well-known and well-tested pose graph optimization library. A `FactorGraph` was constructed from the existing `PoseGraph` data structure and passed to GTSAM's Levenberg–Marquardt optimizer. The optimized poses were then written back into the graph, and the point clouds of each node were reprojected at the updated pose coordinates to remove map smear accumulated from drift.

Similar to loop closure, pose graph optimization is computationally heavy. It is therefore gated to a minimum of 10 keyframes since the last optimization run.

Initial attempts produced unexpected graph translations and, most notably, no improvement to mapping — smear was not removed. This led to a thorough evaluation of the full SLAM pipeline, including implementation of point-to-plane correspondence matching in ICP and iterative hyperparameter refinement. The Webots lidar rotation issue (Section 3.4) was discovered during this investigation, and resolving it was the key breakthrough that brought the full system together.

9. Results

This project produced a working and reliable SLAM implementation able to localize the trajectory of the robot and provide an accurate occupancy grid map of its surroundings within the testing environment.

Creating an entire SLAM system from scratch, without prior experience in the field, is a substantial undertaking that tested system architecture skills and the ability to design, implement, and debug complex code with direct real-world applications.

9.1 Trajectory Accuracy

The success of the system is clearly visible in the trajectory results. ICP demonstrably uses lidar information to calculate trajectories that smooth and correct the errors introduced by noisy odometry.

In the image below we can see the remarkable accuracy of ICP and our Pose Graph in trajectory mapping. We have ICP trajectory in green, Odometry trajectory in blue, ground truth trajectory in white. As well as Pose Graph nodes in pink with their ground truth transformation in a dark pink. We see that ICP in green actually hides the ground truth trajectory, shown in white, only visible at some spots, which goes to show that the estimated trajectory is virtually identical to the ground truth trajectory. We see a slight shift between the Pose Graph nodes and ICP, and this is a result of our Pose Graph Optimization.

Figure 9.1: Final Trajectories and Pose Graph in simulation environment

9.2 Mapping Accuracy

The improvement in map quality from using ICP over raw odometry is clearly visible in the image below. We see how ICP does not smear like odometry does

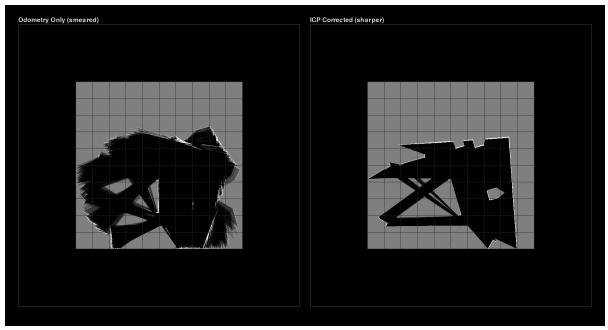
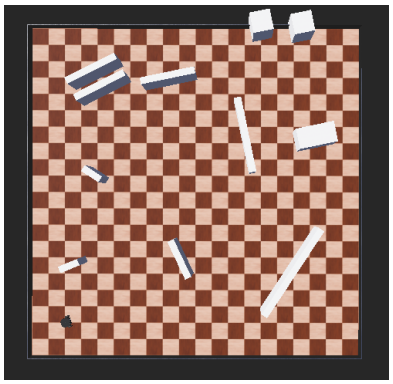
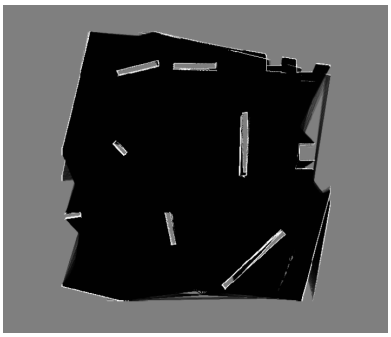


Figure 9.2: In progress occupancy grid from Odometry(left) and ICP(right)

And when we visualize the final output of our occupancy grid compared to our actual simulation environment, we see the remarkable success of the SLAM system.



(a) Simulation environment ground truth.



(b) Output occupancy grid.

Figure 9.3: Ground truth vs. SLAM occupancy grid output for the simulation environment.

9.3 Pose Graph Optimization

Given the small map size ($\approx 2 \times 2$ m), insufficient drift accumulates to produce a visually dramatic graph snap. The improvement can, however, be quantified through the reduction in total graph error before and after optimization. Table 9.1 presents results from several test runs in the simulation environment.

Table 9.1: Pose graph optimization results across multiple test runs.

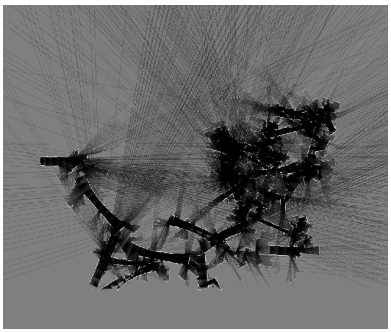
Run	Pre-optimization error (m)	Post-optimization error (m)	Reduction (m)
1	2.07447	1.95095	0.12353
2	0.03138	0.00431	0.02707
3	0.01506	0.00806	0.00700
4	0.00348	0.00261	0.00086

9.4 Validation on a Real-World Dataset

To validate the system beyond the simulation environment, the Intel Research Lab dataset was used — a popular benchmark for 2D SLAM, recorded in a real research facility.This data came from Cyrill Stachniss TODO add reference

As expected, the weaknesses of the system became apparent at this scale. The system was designed for a 2×2 m Webots grid, while the Intel dataset covers a facility over 80 m long with a different lidar scanner and motion profile. Nonetheless, some structural features of the facility are discernible in the output, and loop closure and graph optimization did trigger during replay.

(a) Actual Intel Research Lab.



(b) Output of SLAM system.

Figure 9.4: Ground truth vs. SLAM output on the Intel Research Lab dataset.

Production-scale SLAM systems span entire fields of research and take teams of researchers years to implement and refine. The results of this project are therefore a commendable achievement given the time constraints, lack of prior experience, and available resources.

9.5 Future Work

Several directions could improve the system:

- **Point-to-plane correspondence matching.** Minimizing the distance from a source point to the plane containing its corresponding target point is more robust to noise than point-to-point matching, particularly in planar environments.
- **Sensor fusion with an Extended Kalman Filter.** An EKF could intelligently weight contributions from odometry and lidar, especially during rotations where ICP alone performs poorly.
- **Scaling to larger environments.** All parameters and data structures are tuned for a small, controlled arena. Adapting to larger maps would require algorithmic improvements to loop closure candidate search, correspondence matching, and optimization.

References

References to be added.

A. Representative Source Code

Source code for this project is submitted in electronic format alongside this report. The source code can be found at Github Repository

Code snippets to be added.